
Towards Mitigating the Reversal Curse in LLMs

David Bai Allison Lim

1. Abstract

The Reversal Curse, a reasoning failure in LLMs, describes their inability to retrieve facts in the opposite direction of the way they were learned. We investigate this using a synthetic dataset of character relationships and test three mitigation methods: word-level reversal, entity-level reversal, and our proposed method, train-time augmentation (TTA). Our experiments with Qwen 2.5 7B Instruct demonstrate that TTA outperforms entity-level reversal and word-level reversal, achieving 95% backwards accuracy compared to 43% and 2% accuracy respectively, while our baseline achieved 0%. TTA uses an LLM at train-time to handle data sparsity and uneven distribution by dynamically making inferences about each training sample. Ultimately, these results demonstrate that diverse and bidirectional training examples during fine-tuning can mitigate the Reversal Curse, acting as a workaround to the decoder-only architecture that is the backbone of most modern LLMs. All code can be found here: <https://github.com/davldbai/reversalcurse>.

2. Introduction

Large Language Models (LLMs), specifically autoregressive transformer-based architectures, have demonstrated remarkable capabilities across diverse tasks through scaling. Current frontier models now outperform average humans in mathematics, coding, and depending on the judge, creative domains such as writing. Through sufficient training, these models develop basic logical reasoning abilities, exemplified by emergent circuits such as induction heads (Olsson et al., 2022).

However, these decoder-only models face a fundamental limitation: they generate tokens exclusively based on preceding context. This unidirectional sampling constraint, without external tools like Internet search leads to a significant reasoning failure known as the Reversal Curse (Berglund et al., 2024). The Reversal Curse occurs when models cannot correctly retrieve information in the reverse direction of how they learned it. As defined by Berglund et al. (Berglund et al., 2024), a model trained on the fact "A is [some relationship] to B" struggles to correctly answer questions about "B is [some relationship] to A." For instance, a model trained on "Valentina Tereshkova was the first woman to

travel to space" incorrectly answers the question "Who was the first woman to travel to space?"

To address this limitation, we explore reverse training methods from "Training to Nurse the Reversal Curse", which involves augmenting the original training data with reversed examples (Golovneva et al., 2024). Specifically, we test two variants: word-level reversal (reversing the word order of training examples) and entity-level reversal (preserving entities while reversing their relationships). This approach aims to provide the model with explicit experience processing relationships in both directions, potentially mitigating the curse without architectural changes. Additionally, we propose and test a third approach: Train-Time Augmentation, which uses a LLM to generate more training data using rephrasings and reversals. Our results show that this approach achieves 95% backward accuracy for completions, significantly outperforming other methods.

3. Related Work

The Reversal Curse emerges from the unidirectional nature of autoregressive language models. Researchers attribute this limitation to the next-token prediction objective, which restricts models to considering only preceding context. Lin et al. found that LLMs can generalize to "B is A" relationships when both A and B appear in the context (as in multiple-choice questions), but this ability heavily depends on training document structure (Lin et al., 2024).

Berglund et al. demonstrated that the curse persists across model families and sizes, noting that while useful information typically appears in multiple orders throughout training, Zipf's law ensures many facts appear rarely or only in one direction. This creates persistent blind spots in language models' knowledge.

Studies have found that bidirectional models like BERT appear immune to the Reversal Curse due to their architecture (Wu et al., 2024). Building on this insight, Lv et al. proposed BICO (BIdirectional Causal language modeling Optimization), which modifies traditional LLMs' attention function to be bidirectional (Lv et al., 2024). This approach enables models to consider both preceding and succeeding tokens when making predictions. However, these relies on modifying the inherent mechanisms behind decoder models rather than addressing the problem while maintaining an autoregressive architecture.

There are several methods that explore how to alter the pre-training stage to solve the Reversal Curse. Golovneva et al. introduced "reverse training," which doubles the effective training tokens by presenting inputs in both forward and reverse directions while preserving entities and other key substrings. This method enables bidirectional learning without architectural changes. Guo et al. developed Semantic-aware Permutation Training (SPT), which addresses the root cause of the Reversal Curse by segmenting training sentences into semantic units with an assistant language model before permuting these units (Guo et al., 2024). Unlike simple word-level permutation, SPT maintains the integrity of entities and phrases, facilitating better bidirectional understanding and showing superior performance compared to both standard training and BICO approaches.

Our method was inspired by a previous work, Reasoning to Learn from Latent Thought (Ruan et al., 2025), which models latent "thoughts" from a LLM as a way to create synthetic data from singular training examples. While their method requires training a model to infer information from a compressed datapoint, our method is training-free, leveraging the strengths of existing pre-trained models. A concurrent work we discovered after ideating this method is Augmented Finetuning, from "On the generalization of language models from in-context learning and finetuning: a controlled study" (Lampinen et al., 2025), which employs a very similar method to ours of using a LLM to infer more data from training examples, and also reports results on the Reversal Curse.

4. Dataset and Evaluation

To establish a baseline and reproduce the phenomenon of the Reversal Curse, as well as test our methods, we created a synthetic dataset inspired by the BioS and iGSM8K datasets (Allen-Zhu & Li, 2024a; Ye et al., 2024). This allows us to scale up its size and complexity for different experiments, and lowers the chances of data contamination when we post-train on models with undisclosed pre-training corpora.

Inspired by the biographical nature of the initial reversal curse studies, our dataset consists of character-character relations between fictitious characters. We generate the dataset by creating a graph of characters and relations, then using this graph, generate a training set and backwards and forward test sets. We first describe how this graph is generated.

Given a list C of characters and a list R of relationships — we can sample C_i , and C_j (e.g. "Liam O'Connor" and "Malik Johnson"), as well as R_i where R_i is a pair of asymmetric relationship words (e.g doctor/patient instead of friend/friend). This allows us to form a forward and backward relation as following:

Forward: Liam O'Connor is the doctor of Malik Johnson

Backward: Malik Johnson is the patient of Liam O'Connor

Each character can have multiple relationships to other characters in C .

For this dataset, we support generation in a question-answer (QA) format as well as a completion format:

QA: Who is Liam O'Connor's patient? Malik Johnson

Completion: Liam O'Connor's patient is Malik Johnson

We use accuracy on sequence generation from the model as our evaluation metric. For QA, we measure accuracy by an exact string match in generation and for completions we measure for accuracy by the presence of the answer string in the completion.

Here is an example of what the train set as well as forwards and backwards test sets look like:

Table 1. Training and Testing Formats

QA Format	
Training:	Input: Which student is taught by Nadia Abadi?
	Output: Omar Hassan
	Input: Who studies under Nadia Abadi?
Testing:	Output: Omar Hassan
	Forward: Who is Nadia Abadi's student?
	Expected: Omar Hassan
Testing:	Backward: Who is Omar Hassan's principal?
	Expected: Nadia Abadi
Completion Format	
Training:	Input: The student associated with Nadia Abadi is
	Output: Omar Hassan
	Input: Nadia Abadi has a student by the name of
Testing:	Output: Omar Hassan
	Forward: Nadia Abadi's student is
	Expected: Omar Hassan
Testing:	Backward: Omar Hassan's principal is
	Expected: Nadia Abadi

In order to determine the exact dataset we use to report our results, we test several ablations, measuring differences in performance on the forward test set. We determine whether the complexity of the graph (Fig 1, Fig 2), as well as whether constricting gendered relations, shown in table 2, to labeled names affects performance. More specifically, we compare performance for unique-matching pairs of relationships [2] to a randomized graph of relationships [1], and gendered relationship generation to ungendered relationship generation. For all ablations, we utilize the same set of characters and relations, we simply change how we sample from them. For the gendered dataset, instead of sampling from a full set of characters for a given relation, we constrain the available set of source and target characters to those tagged with the same gender as that relation. For the ungendered dataset,

we do not enforce this constraint. We use GPT-4o to label these relations and characters (OpenAI et al., 2024).

	Gendered	Ungendered
Relationship	sister (f) ↔ brother (m)	uncle (m) ↔ niece (f)
Characters	Aleksandra Ivanova(f), Gabriel Carter (m)	Akira Tanaka (f), Julian Arnold (m)
Example	Who is Aleksandra Ivanova’s brother?, Gabriel Carter.	Who is Akira Tanaka’s niece?, Julian Arnold

Table 2. Characters and relations can be tagged as f (female), m (male), or n (neutral)

We define a complex graph as any graph where a character may have more than one relationship with other characters. To form complex graphs, we randomly sample 1 to N relations per character. For a simple graph, each character only has one relation — forming a bipartite graph.

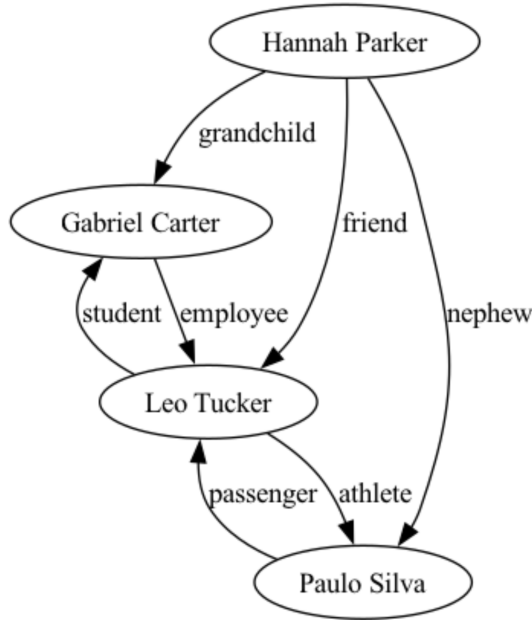


Figure 1. Example Complex Graph

We explain the full details and report performance metrics per dataset variation in Section 6. All in all, we test 4 dataset variants per format: Simple-Gendered, Simple-Ungendered, Complex-Gendered, and Complex-Ungendered.

We choose Simple-Ungendered as our dataset variation to continue experiments on because it demonstrates the largest

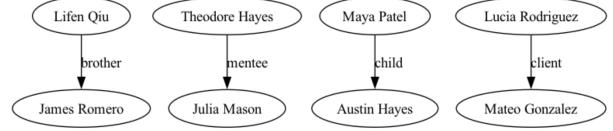


Figure 2. Example Simple Graph

difference in between forward and backward performance, as well as best forward accuracy, across both formats. Other variants with better backward accuracy or lower forward accuracy may have led to confounding variables—the focus of our paper is not on this initial dataset curation or fine tuning, but rather methods that can be applied on top of these processes to mitigate the Reversal Curse. We create 100 relationships and have 4 paraphrases per relationship, creating a training set of size 400.

5. Methods

Because we are evaluating a problem that persists across most autoregressive, transformer-based models, we have the option of either pre-training or finetuning our baseline model on our dataset. We considered pre-training, but ultimately decided on finetuning due to computational limitations — the memory footprint of a full finetune on a 7B-size model was too large for our machines.

For our baseline and methods, the model we chose to fine-tune was Qwen 2.5 7B Instruct (Qwen et al., 2025), which we provide a justification for compute and performance-wise for in Section 6. Besides fitting our experiment bounds, we also chose the Qwen family because it demonstrates “cognitive behaviors” that make it more amenable to post-training for extensive chain-of-thought and reasoning (Gandhi et al., 2025), a method we hope to try in the future. We also believe showing that this problem applies to current models is important, and (at time of writing), this model had 2.1M downloads on Hugging Face in the last month, with several models from Qwen trending— making it a plausible representation of current model capabilities.

Our current baseline is Qwen 2.5 7B Instruct fine-tuned on our forward training data and its performance on our backward test set.

To mitigate the reversal curse, we employ and test 2 variants of Reverse Training from “Training to Nurse the Reversal Curse” (Golovneva et al., 2024). Reverse training is defined as constructing a set of reversed training examples from an initial training set T, then combining this with the original training set. This produces a new training set with double the size— which we perform our standard finetuning process on. We employ and test two types of reversal operations: word-level and entity level, shown in table 3.

None	The patient associated with Liam O'Connor is Malik Johnson
Word Reversal	Johnson Malik is O'Connor Liam with associated patient The
Entity Reversal	Malik Johnson is Liam O'Connor with associated patient The

Table 3. Types of sentence transformations

For word-level reversal, we simply separate by spaces and reverse the sequence— for entity reversal, we use `en_core_web_sm`, a small English language model, to detect entities and consider those one word when separating (Honnibal et al., 2020).

We also introduce our own method, Training-Time Augmentation, which addresses the limitations of the previous 2 methods. Unlike word-level and entity-level reversal, which apply a static transformation, this method provides varied sentences that typically capture the bidirectional relationship. Notably, this is also the only method used that augments the data in a grammatically correct manner. Additionally, while word-level and entity-level reversal are difficult to apply outside of a completion format, fine-tuning with TTA can be applied to both completion and QA.

For each example in our training set, we use an LLM (GPT-4o-mini) to dynamically extract and infer additional knowledge from the limited training samples. This inference-based approach generates new training examples, shown in table 4, by identifying both alternative expressions of the same relationship (maintaining directionality) and logical deductions about the inverse relationships between entities. By prompting the model to make these cognitive inferences about the underlying knowledge representation, we create a richer, more diverse training set that captures the relational information from multiple perspectives. This train-time inference approach significantly expanded our dataset - for the Simple-Ungendered-Completion format, the dataset grew from 400 to 2243 examples, and for the Simple-Ungendered-QA format, from 400 to 2055 examples.

TTA overcomes the limitations of other dataset permutation/enrichment methods by leveraging compute to create well-formed, diverse, training examples that can be applied to multiple completion formats, and the reasoning capabilities of LLMs to infer, in our case, inverse relationships, but more broadly any other implicit facts defined in the training data.

To properly compare the efficacy of these methods, we hold hyperparameters, etc. constant— for clarity, these are described in our experiment section after we justify our dataset, etc.

Completion Format	
Type	Example
Original	Liam O'Connor is the patient of Eva Phillips
Rephrasing	Liam O'Connor receives care from Eva Phillips
Reversal	Eva Phillips is the healthcare provider for Liam O'Connor
QA Format	
Type	Example
Original	Q: Who is Eva Phillips's patient? A: Liam O'Connor
Rephrasing	Q: Which individual receives medical treatment from Eva Phillips? A: Liam O'Connor
Reversal	Q: Who provides healthcare to Liam O'Connor? A: Eva Phillips

Table 4. Examples of inferences from Train-Time Augmentation

6. Experiments

To test our methods and establish our baseline, we needed to first establish a working setup for model fine-tuning. We trained the 0.5B and 3B variants of Qwen 2.5 in Google Colab using quantization with Unsloth— but, at least with our cursory experimentation, found difficulty learning our QA format. Our primary rationale for increasing parameter size instead of, say, switching model families, came from prior work demonstrating that knowledge storage capacity in a model scales with parameter count (Allen-Zhu & Li, 2024b).

All these preliminary experiments were performed on solely a QA dataset, motivated by this work which demonstrates that QA data is superior to other forms of training data for knowledge acquisition (Zhao et al., 2025).

We ultimately decided to train Qwen2.5 7B Instruct using LoRa finetuning. We initially did some informal experimentation, described below, to find a stable training setup, then used that to identify our target dataset type, and then tuned our final hyperparameters on that dataset type. We then report results for that dataset.

6.1. Initial Training Setup

When experimenting with finetuning, we initially found poor results on our Simple-Gendered QA dataset variant, and tried various methods such as scaling up training set size (examples per relationship), ablating chat template for QA format, using the base model directly, and adjusting hyperparameters such as learning rate. While increasing the diversity and size of the training set helped (10% im-

provement from 25% to 35%), we ultimately scaled up LoRa rank, keeping other hyperparameters to the Hugging-Face SFT defaults. For training on Simple-Gendered QA, we used a LoRa rank of 512, which corresponds to about 16% of parameters being trainable (compared to just 2% for rank 64)— and regularly trained to a high accuracy on our Simple-Gendered QA dataset. For training on a completion format instead of QA, we end up using a rank of 1024 (27% of parameters trainable)— we attribute this relatively inefficient knowledge storage, compared to QA, based on the findings from Zhao et al.

6.2. Dataset Selection

Finding a sufficiently stable training method, we turn to identifying the specific dataset type we want to evaluate on. We report the average accuracy of Qwen 2.5 7B on each of these dataset variants defined in section 4 after performing supervised fine tuning for 200 steps or 50 epochs, with hyperparameters established in Section 6.1

Dataset	Forward Accuracy	Backward Accuracy
Simple + Ungendered	99	0
Simple + Gendered	97	0
Complex + Ungendered	76.47	2.94
Complex + Gendered	82.83	1.01

Table 5. QA Accuracy

Dataset	Forward Accuracy	Backward Accuracy
Simple + Ungendered	92	0
Simple + Gendered	83	0
Complex + Ungendered	68.63	1.96
Complex + Gendered	71.72	0

Table 6. Completions Accuracy

For consistency, the complex datasets and gender-neutral datasets are adjusted to have around 100 relationships so we can keep this constant and focus on how gender phrasing and complexity affect model performance. To do this, we restrict the complex dataset to 50 unique characters instead of 100 for the simple datasets. The gendered complex dataset comes out to 99 relationships while the ungendered complex dataset comes out to 102— we consider this a negligible difference when evaluating model performance. We utilize the same prompt for generating paraphrases per dataset, though LLM generation is inherently stochastic so all training examples will be slightly different— and generate 4 training examples per relationship.

Averaging across both sequence types, gendered versus ungendered has a minimal difference of 0.39% while complex

versus simple has a difference of 17.59%. In the backward pass, complex has an average accuracy of 1.48 compared to 0 for simple. We attribute this to the fact that in the training data, a single source character can have multiple target characters— when we query about a relation to a target character, the chance of a random source character being the correct choice is slightly higher— combined with the smaller character count for our complex datasets.

6.3. Hyperparameter Tuning

Having established a specific dataset type, we can now tune our hyperparameters without having to test across all 8 dataset variants.

To ensure unbiased evaluation, hyperparameter optimization was conducted using a development dataset. This dataset, comprising training and validation splits, was generated from an initial graph of character relationships. The final reported results (Section 6.4) utilize a separate, held-out evaluation graph. From this evaluation graph, we constructed a fine-tuning set (containing only forward-direction facts for the baseline) and corresponding forward and backward test sets. All methods were trained on their respective fine-tuning sets (augmented for TTA and reverse training, baseline trained on forward facts only) and subsequently evaluated on these test sets to measure their ability to handle the Reversal Curse.

We first tune hyperparameters for our train set with a LoRa rank of 512 (sourced from initial experiments), arriving at the following in table 7:

Parameter	Value
learning_rate	1e-5
optim	adamw
weight_decay	0.01
warmup_ratio	0.1
lr_scheduler_type	cosine
per_device_train_batch_size	50
gradient_accumulation_steps	2

Table 7. Training Hyperparameters

Our main changes were to the per device training batch, for which we found increasing improved performance (though we stopped at 50 to keep training time reasonable), and to the optimizer, for which we found adamw to perform better than adam. We then hold these parameters constant while scaling up LoRa rank[8]— we find ranks of 2^n for $n \leq 6$ fail to accurately learn any relations.

Having confirmed the LoRa ranks we used in Section 6.1 to be optimal (with ranks > 1024 exceeding our GPU memory and thus not testable), we finalize our hyperparameters, reporting test set metrics in Section 6.4.

Rank	Completions	QA
64	0.00	0.00
128	65.00	57.00
256	73.00	87.00
512	83.00	100.00
1024	90.00	100.00

Table 8. Forward Accuracy (%) by LoRA Rank

6.4. Method Results

We report the forward and backward accuracy of our model fine-tuned with the baseline training set on the Simple-Gendered-Completion dataset, and our model fine-tuned with the reverse word and reverse entity methods, all for 200 steps.

Method	Forward Accuracy	Backward Accuracy
Baseline	91%	1%
Reverse Word	93%	2%
Reverse Entity	93%	43%
Train-Time Augmentation	100%	95%

We find that all training methods achieve similar levels of accuracy on the forward test set— while backward accuracy only improves with the Reverse Entity method. We speculate this is because our characters have a first and last name. For a dataset with one-word characters, Reverse Word would be functionally the same as Reverse Entity and we would expect to see a performance gain in backward accuracy as a result.

We speculate the improvement from TTA that is not present with Reverse Entity happens for several reasons. Since TTA maintains grammatical correctness, it helps the model learn relationships over syntax patterns. Reverse Entity preserves entities, but as shown in Table 2, does not reflect a true reversal of the relationship. Essentially, TTA explicitly teaches the model bidirectionality in relationships. For completions, the TTA Dataset (2243 examples) is also larger than the Reverse Entity one (800 examples).

We also report the forward and backward accuracy of our model fine-tuned with the baseline training set on the Simple-Ungendered-QA dataset, and our model fine-tuned with the TTA method, all for 200 steps.

Method	Forward Accuracy	Backward Accuracy
Baseline	100%	0%
Train-Time Augmentation	99%	93%

7. Discussion

For both the forward and backward test sets, we report 2 error metrics as defined below:

In-Domain Entity Error: The proportion of the erroneous character answers that were characters in the training dataset.

Relation-Preserving Error: The proportion of erroneous character answers that answered with a character that has the same relationship assignment (e.g. A and B, as well as C and D both have a mother-child relationship, and a query on A is answered with D).

Method	In-Domain Entity Error	Relationship Preserving Error
Baseline Forward	8/9	4/9
Baseline Backward	82/99	2/99
Reversed Word Forward	6/7	1/7
Reversed Word Backward	66/98	1/98
Reversed Entity Forward	6/7	0/7
Reversed Entity Backward	35/57	0/57
Train-Time Augmentation Forward	0	0
Train-Time Augmentation Backward	4/5	0/5

Table 9. Error analysis for Completion Format

Method	In-Domain Entity Error	Relationship Preserving Error
Baseline Forward	0	0
Baseline Backward	69/98	0/98
Train-Time Augmentation Forward	1/1	0/1
Train-Time Augmentation Backward	7/7	0/7

Table 10. Error analysis for QA Format

To provide some concrete examples of error types: For the relationship where Alexander Reed is Jinhai Ruan’s employee and Jinhai Ruan is Alexander Reed’s boss, an example of an in-domain entity error is the model outputting “named Xiuying Gao” when prompted with “Who is Jinhai Ruan’s employee?” where Xiuying Gao is a character present in the dataset. Given a bidirectional relationship where Warren Lyons and Willow Byrd are friends, an example of a relationship preserving error is the model outputting

“Christian Perry” when prompted with “Who is Warren Lyons’s friend?” Christian Perry is Isaac Cohen’s friend; thus, this is another friend/friend relationship. We do not detect any common hallucination patterns besides the two types of errors we defined.

We find that regardless of method, model answers are primarily in-domain entities. For our completion format baseline, 8/9 forward errors and 82/99 backward errors involve in-domain characters. Across all methods, backward test sets have higher in-domain error rates relative to their total errors, which could indicate that they have strong memorization of the characters but weak understanding of relationships in reverse. TTA on the backward test set has the highest in-domain entity error (4/5 for completion, 7/7 for QA), which indicates it has learned to constrain its answers to the character pool well rather than hallucinating names. Relationship preserving error was minimal across backward tests and higher on forward tests.

We are also interested in whether or not the specific relation ascribed affects the model’s ability to learn a given relationship. We analyze the number of errors per relation and report errors as a percentage of all appearances of the same relationship in the test set. For readability, we do not present relations with 0 errors across all methods.

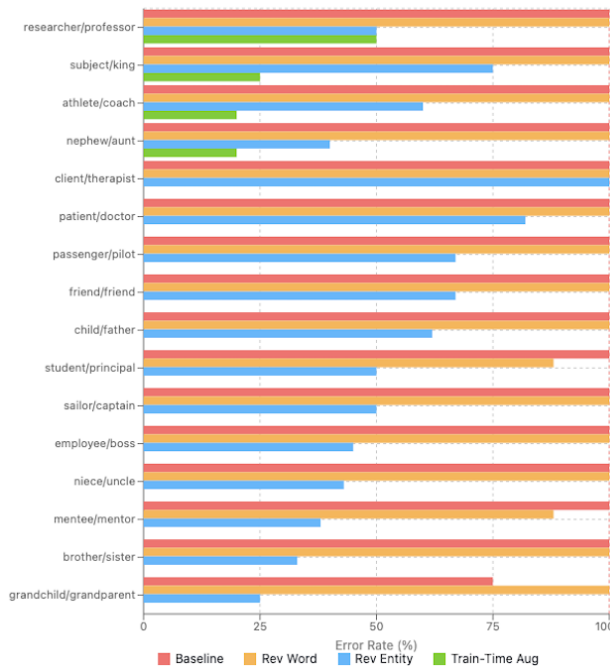


Figure 3. Errors per relation across methods on backward test set

Across methods, we do not find a clear pattern in what relations methods perform well or poorly on. However, we do see there is high variance across relationship types — implying that this plays some role in performance.

While Baseline and Reverse Word have nearly 100% error on the backward test set across all relationship types, TTA reduces errors to 0% for most relationship types other than the following: researcher/professor, subject/king, athlete/coach and nephew/aunt. We note that the relationships TTA struggles with are a subset of those Reverse Entity struggles with, possibly indicating that certain relationships are more difficult to learn the opposite direction in.

We do not perform any ablation studies with character names, so we lack sufficient evidence to claim whether these relation types are the sole factor that affects method accuracy. In future, we would like to investigate this further but find it out of scope for this immediate work.

8. Conclusion

In this report, we propose a dataset generation method for testing the reversal curse with character-character relations, as well as demonstrate how variations in dataset creation affect model performance. We establish that the reversal curse exists for knowledge acquired via finetuning on Qwen 2.5 7B Instruct, and explore previously proposed methods for mitigating the reversal curse. Our proposed Train-Time Augmentation (TTA) method significantly outperforms other approaches, achieving up to 95% backward accuracy by leveraging an LLM to dynamically generate training examples that capture bidirectional relationships. Moving forward, we’d like to evaluate the performance of non- or semi-autoregressive language models on this dataset, like BERT, which has bidirectional attention, or LLaDA-8B, a novel text diffusion model as additional baselines (Devlin et al., 2019; Nie et al., 2025).

Future methods we would like to try include an emphasis on knowledge retrieval in order to retrieve relevant entities for reverse relations— putting them in-context for the model to attend to. In preliminary experiments with Qwen 2.5 7B Instruct, we found that further prompting the model about a character from our dataset led to confabulations about biographical history, references to TV shows, etc. To alleviate this, we plan on analyzing a model with an open source training corpus— like from the Olmo2 family, and extracting our QA/completions dataset from there, on a set of celebrities — preventing knowledge acquisition issues with finetuning, and hallucinations.

References

- Allen-Zhu, Z. and Li, Y. Physics of language models: Part 3.1, knowledge storage and extraction, 2024a. URL <https://arxiv.org/abs/2309.14316>.
- Allen-Zhu, Z. and Li, Y. Physics of language models: Part 3.3, knowledge capacity scaling laws, 2024b. URL

<https://arxiv.org/abs/2404.05405>.

Berglund, L., Tong, M., Kaufmann, M., Balesni, M., Stickland, A. C., Korbak, T., and Evans, O. The reversal curse: Lms trained on "a is b" fail to learn "b is a", 2024. URL <https://arxiv.org/abs/2309.12288>.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.

Gandhi, K., Chakravarthy, A., Singh, A., Lile, N., and Goodman, N. D. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars, 2025. URL <https://arxiv.org/abs/2503.01307>.

Golovneva, O., Allen-Zhu, Z., Weston, J., and Sukhbaatar, S. Reverse training to nurse the reversal curse, 2024. URL <https://arxiv.org/abs/2403.13799>.

Guo, Q., Wang, R., Guo, J., Tan, X., Bian, J., and Yang, Y. Mitigating reversal curse in large language models via semantic-aware permutation training, 2024. URL <https://arxiv.org/abs/2403.00758>.

Honnibal, M., Montani, I., Van Landeghem, S., and Boyd, A. spaCy: Industrial-strength Natural Language Processing in Python. 2020. doi: 10.5281/zenodo.1212303.

Lampinen, A. K., Chaudhry, A., Chan, S. C. Y., Wild, C., Wan, D., Ku, A., Bornschein, J., Pascanu, R., Shannah, M., and McClelland, J. L. On the generalization of language models from in-context learning and finetuning: a controlled study, 2025. URL <https://arxiv.org/abs/2505.00661>.

Lin, Z., Fu, Z., Liu, K., Xie, L., Lin, B., Wang, W., Cai, D., Wu, Y., and Ye, J. Delving into the reversal curse: How far can large language models generalize?, 2024. URL <https://arxiv.org/abs/2410.18808>.

Lv, A., Zhang, K., Xie, S., Tu, Q., Chen, Y., Wen, J.-R., and Yan, R. An analysis and mitigation of the reversal curse, 2024. URL <https://arxiv.org/abs/2311.07468>.

Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models, 2025. URL <https://arxiv.org/abs/2502.09992>.

Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J.,

Kaplan, J., McCandlish, S., and Olah, C. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>.

OpenAI, :, Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., Madry, A., Baker-Whitcomb, A., Beutel, A., Borzunov, A., Carney, A., Chow, A., Kirillov, A., Nichol, A., Paino, A., Renzin, A., Passos, A. T., Kirillov, A., Christakis, A., Conneau, A., Kamali, A., Jabri, A., Moyer, A., Tam, A., Crookes, A., Tootoochian, A., Tootoonchian, A., Kumar, A., Vallone, A., Karpathy, A., Braunstein, A., Cann, A., Codisopoti, A., Galu, A., Kondrich, A., Tulloch, A., Mishchenko, A., Baek, A., Jiang, A., Pelisse, A., Woodford, A., Gosalia, A., Dhar, A., Pantuliano, A., Nayak, A., Oliver, A., Zoph, B., Ghorbani, B., Leimberger, B., Rossen, B., Sokolowsky, B., Wang, B., Zweig, B., Hoover, B., Samic, B., McGrew, B., Spero, B., Giertler, B., Cheng, B., Lightcap, B., Walkin, B., Quinn, B., Guarraci, B., Hsu, B., Kellogg, B., Eastman, B., Lugaresi, C., Wainwright, C., Bassin, C., Hudson, C., Chu, C., Nelson, C., Li, C., Shern, C. J., Conger, C., Barette, C., Voss, C., Ding, C., Lu, C., Zhang, C., Beaumont, C., Hallacy, C., Koch, C., Gibson, C., Kim, C., Choi, C., McLeavey, C., Hesse, C., Fischer, C., Winter, C., Czarnecki, C., Jarvis, C., Wei, C., Koumouzelis, C., Sherburn, D., Kappler, D., Levin, D., Levy, D., Carr, D., Farhi, D., Mely, D., Robinson, D., Sasaki, D., Jin, D., Valladares, D., Tsipras, D., Li, D., Nguyen, D. P., Findlay, D., Oiwoh, E., Wong, E., Asdar, E., Proehl, E., Yang, E., Antonow, E., Kramer, E., Peterson, E., Sigler, E., Wallace, E., Brevdo, E., Mays, E., Khorasani, F., Such, F. P., Raso, F., Zhang, F., von Lohmann, F., Sulit, F., Goh, G., Oden, G., Salmon, G., Starace, G., Brockman, G., Salman, H., Bao, H., Hu, H., Wong, H., Wang, H., Schmidt, H., Whitney, H., Jun, H., Kirchner, H., de Oliveira Pinto, H. P., Ren, H., Chang, H., Chung, H. W., Kivlichan, I., O’Connell, I., O’Connell, I., Osband, I., Silber, I., Sohl, I., Okuyucu, I., Lan, I., Kostrikov, I., Sutskever, I., Kanitscheider, I., Gulrajani, I., Coxon, J., Menick, J., Pachocki, J., Aung, J., Betker, J., Crooks, J., Lennon, J., Kiros, J., Leike, J., Park, J., Kwon, J., Phang, J., Teplitz, J., Wei, J., Wolfe, J., Chen, J., Harris, J., Varavva, J., Lee, J. G., Shieh, J., Lin, J., Yu, J., Weng, J., Tang, J., Yu, J., Jang, J., Candela, J. Q., Beutler, J., Landers, J., Parish, J., Heidecke, J., Schulman, J., Lachman, J., McKay, J., Uesato, J., Ward, J., Kim, J. W., Huizinga, J., Sitkin, J., Kraaijeveld, J., Gross, J., Kaplan, J., Snyder, J., Achiam, J., Jiao, J., Lee, J., Zhuang, J., Harriman, J., Fricke, K., Hayashi, K., Singhal, K., Shi, K., Karthik, K., Wood, K., Rimbach, K., Hsu, K., Nguyen, K., Gu-Lemberg, K., Button, K., Liu, K., Howe, K., Muthukumar, K., Luther, K., Ahmad, L., Kai, L., Itow, L., Workman, L., Pathak, L., Chen, L., Jing, L., Guy, L.,

-
- Fedus, L., Zhou, L., Mamitsuka, L., Weng, L., McCalum, L., Held, L., Ouyang, L., Feuvrier, L., Zhang, L., Kondraciuk, L., Kaiser, L., Hewitt, L., Metz, L., Doshi, L., Aflak, M., Simens, M., Boyd, M., Thompson, M., Dukhan, M., Chen, M., Gray, M., Hudnall, M., Zhang, M., Aljube, M., Litwin, M., Zeng, M., Johnson, M., Shetty, M., Gupta, M., Shah, M., Yatbaz, M., Yang, M. J., Zhong, M., Glaese, M., Chen, M., Janner, M., Lampe, M., Petrov, M., Wu, M., Wang, M., Fradin, M., Pokrass, M., Castro, M., de Castro, M. O. T., Pavlov, M., Brundage, M., Wang, M., Khan, M., Murati, M., Bavarian, M., Lin, M., Yesildal, M., Soto, N., Gimelshein, N., Cone, N., Staudacher, N., Summers, N., LaFontaine, N., Chowdhury, N., Ryder, N., Stathas, N., Turley, N., Tezak, N., Felix, N., Kudige, N., Keskar, N., Deutsch, N., Bundick, N., Puckett, N., Nachum, O., Okelola, O., Boiko, O., Murk, O., Jaffe, O., Watkins, O., Godement, O., Campbell-Moore, O., Chao, P., McMillan, P., Belov, P., Su, P., Bak, P., Bakkum, P., Deng, P., Dolan, P., Hoeschele, P., Welinder, P., Tillet, P., Pronin, P., Tillet, P., Dhariwal, P., Yuan, Q., Dias, R., Lim, R., Arora, R., Troll, R., Lin, R., Lopes, R. G., Puri, R., Miyara, R., Leike, R., Gaubert, R., Zamani, R., Wang, R., Donnelly, R., Honsby, R., Smith, R., Sahai, R., Ramchandani, R., Huet, R., Carmichael, R., Zellers, R., Chen, R., Chen, R., Nigmatullin, R., Cheu, R., Jain, S., Altman, S., Schoenholz, S., Toizer, S., Miserendino, S., Agarwal, S., Culver, S., Ethersmith, S., Gray, S., Grove, S., Metzger, S., Hermani, S., Jain, S., Zhao, S., Wu, S., Jomoto, S., Wu, S., Shuaiqi, X., Phene, S., Papay, S., Narayanan, S., Coffey, S., Lee, S., Hall, S., Balaji, S., Broda, T., Stramer, T., Xu, T., Gogineni, T., Christianson, T., Sanders, T., Patwardhan, T., Cunningham, T., Degry, T., Dimson, T., Raoux, T., Shadwell, T., Zheng, T., Underwood, T., Markov, T., Sherbakov, T., Rubin, T., Stasi, T., Kaftan, T., Heywood, T., Peterson, T., Walters, T., Eloundou, T., Qi, V., Moeller, V., Monaco, V., Kuo, V., Fomenko, V., Chang, W., Zheng, W., Zhou, W., Manassra, W., Sheu, W., Zaremba, W., Patil, Y., Qian, Y., Kim, Y., Cheng, Y., Zhang, Y., He, Y., Zhang, Y., Jin, Y., Dai, Y., and Malkov, Y. Gpt-4o system card, 2024. URL <https://arxiv.org/abs/2410.21276>.
- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Ruan, Y., Band, N., Maddison, C. J., and Hashimoto, T. Reasoning to learn from latent thoughts, 2025. URL <https://arxiv.org/abs/2503.18866>.
- Wu, D., Yang, J., and Wang, K. Exploring the reversal curse and other deductive logical reasoning in bert and gpt-based large language models, 2024. URL <https://arxiv.org/abs/2312.03633>.
- Ye, T., Xu, Z., Li, Y., and Allen-Zhu, Z. Physics of language models: Part 2.1, grade-school math and the hidden reasoning process, 2024. URL <https://arxiv.org/abs/2407.20311>.
- Zhao, E., Awasthi, P., and Haghtalab, N. From style to facts: Mapping the boundaries of knowledge injection with finetuning, 2025. URL <https://arxiv.org/abs/2503.05919>.